**SOFTWARE REQUIREMENTS SPECIFICATION (PROJECT IRO)**

**Project Title:** Issues Reporting Outlet (IRO)
**Institution:** Adekunle Ajasin University, Akungba-Akoko
**Course Code:** CSC426
**Document Type:** Software Requirements Specification

## Introduction

### Purpose

This document lays out what the Issues Reporting Outlet (IRO) actually needs to do. At its core, IRO is a web app built to help students and local residents report problems in their community and keep an eye on whether anything is being done about them.

### Scope

Think of IRO as a central hub for tackling community issues around Adekunle Ajasin University. Anyone using the platform can drop a report, grab a reference number, and track how the issue is moving along. On the flip side, admins have a backend where they can look over reports, sort them into categories, update their progress, and log what steps have been taken.

We're focusing mainly on everyday headaches: power outages, water shortages, bad roads, blocked drainage, waste management issues, security worries, and general public facility damage.

To keep things realistic for this version, IRO won't magically fix the problems itself, handle government budgets, or force external agencies to take action. It's strictly a reporting and tracking tool.

### Intended Users

**Community Members:** Students and local residents living around the campus.
**System Admins:** The people in charge of reviewing and sorting through incoming reports.
**Developers:** Ourselves, handling the building and maintenance of the platform.

### Definitions

**IRO:** Issues Reporting Outlet.
**SRS:** Software Requirements Specification.
**User:** Anyone using the web app.
**Administrator:** An authorized user with permission to manage and update reports.
**Report ID:** The unique tracking code given to a submitted issue.
**Status:** Where a particular report currently stands in the resolution process.

## Overall Description

### Product Perspective

IRO is a straightforward web application made up of three main parts: a front-end user interface, a back-end server, and a database. Users just need a web browser to access it. When someone submits a report, the server checks the details and saves everything securely in the database.

### Product Functions

Here is what the system is designed to do:
Show visitors what IRO is all about on the homepage.
Let users submit reports about community issues.
Double-check that all required fields are filled out properly before saving.
Hand out a unique tracking ID for every successful report.
Let people check the live status of their reports using their ID.
Give admins the tools to review, update, and manage submissions.
Keep a log of status updates and actions taken.
Show basic statistics and metrics on submitted reports.

### User Characteristics

We're assuming everyday users know the basics of using a smartphone or laptop and a web browser. Admins, however, will need to know how to log in, prioritize issues, and update statuses correctly.

### Operating Environment

**Client Devices:** Smartphones, tablets, or laptops/PCs.
**Interface:** Any modern web browser (Chrome, Firefox, Safari, etc.).
**Frontend:** HTML, CSS, and JavaScript.
**Backend:** Python (Flask framework).
**Database:** SQLite (for this initial version).
**Development Tools:** Visual Studio Code and Git for version control.

### Constraints

Users need an active internet connection to use the live version.
We have to make sure report data is secure and protected from nosy outsiders.
Only admins are allowed to change the status of a report.
The system relies heavily on users actually typing in accurate information.
Integration with external government or university authorities is out of scope for this version.

### Assumptions and Dependencies

Users actually have a device with a browser and internet access.
Admins will regularly log in and look at the reports coming in.
The server hosting the app won't
randomly crash and will support our database.
People filing reports will give enough detail for the issues to actually make sense.

## Functional Requirements

**FR-01: Display Homepage**
The app must greet visitors with a clear explanation of what IRO does, along with easy-to-find buttons pointing to the report submission and tracking pages.

**FR-02: Submit an Issue**
The system must provide a form where a user can enter a title, pick a category, state the location, and write a description of the problem.

**FR-03: Validate Submissions**
If someone tries to submit a report with empty required fields or gibberish, the system needs to block it and pop up a helpful error message explaining what's missing.

**FR-04: Generate Report Reference**
As soon as a report is successfully saved, the system must spit out a unique reference number so the user can track it later.

**FR-05: Save Reports**
The backend must successfully capture all valid report data, log the exact date it was created, and assign it an initial "Pending" or starting status in the database.

**FR-06: Track Reports**
The system must allow any user to type in their unique reference ID and instantly view the current status and history of their submitted issue.